Data Preprocessing & Feature Engineering

Turn raw Kenyan tourism data into a model-ready dataset that unlocks real business insights.

⏳ 6 hours total✍️ 4 labs + 1 assignment✅ Quiz + Lab + Forum

By end of this week, you will be able to

Identify and handle missing values across MCAR, MAR, and MNAR types
Detect and treat outliers using IQR and Z-score methods
Apply scaling techniques like StandardScaler and MinMaxScaler
Encode categorical variables with label encoding and one-hot encoding
Engineer new features from dates, ratios, and binning
Build a complete preprocessing pipeline with ColumnTransformer and Pipeline

1. Why Preprocessing Matters

Consider a Kenyan microfinance institution that built a credit-scoring model for small business loans. The model rejected women-owned businesses at 2.3x the rate of men-owned ones. Auditors later found a data entry error: some loan officers recorded amounts in Kenyan shillings while others used US dollars, and no one checked. The model learned to penalize any currency value that looked "too high"-which disproportionately affected women-led businesses that listed multiple income sources correctly in shillings.

Most data scientists spend 60–80% of their time cleaning and preparing data. This step separates robust, fair models from ones that fail silently.

Raw Data

→

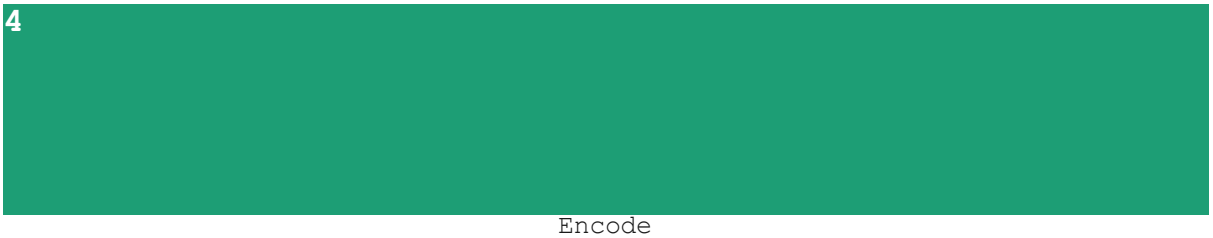
Clean

→

Outliers

→

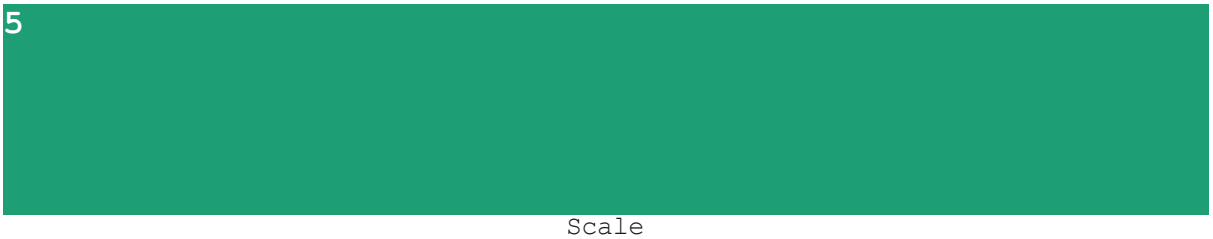
4



Encode

→

5



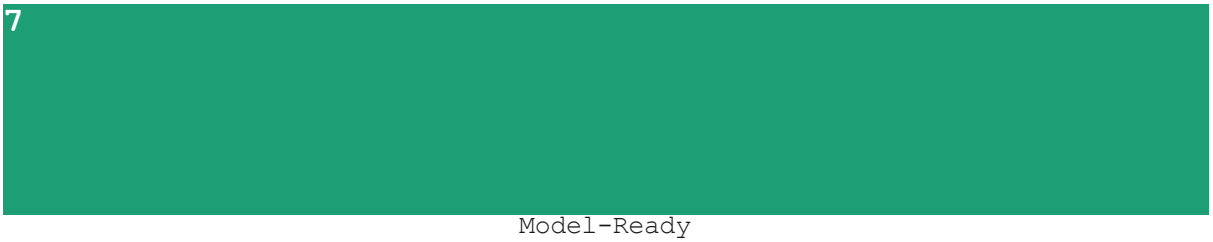
Scale

→

6

Engineer

→



2. Handling Missing Data

Missing data comes in three forms. How you treat it depends on why it is missing.

Type	What it means	Example	Strategy

MCAR	Missing Completely At Random. Pure chance, no pattern.	A survey tablet died mid-interview in Kisumu.	Drop or impute without bias.
MAR	Missing At Random. Related to observed data.	Younger respondents skip income questions in Kenya FinAccess surveys.	Impute using other columns like age, location.
MNAR	Missing Not At Random. Related to the hidden value itself.	High-income tourists in Tanzania hide their spending.	Flag missingness AND impute. Never ignore.

LAB 1: MISSING VALUES ● Run in Colab

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
```

```
# Load Tanzania tourism dataset
df = pd.read_csv("Tanzania_Tourism.csv")

# Visualize percentage missing per column
missing = (df.isnull().sum() / len(df)) * 100
missing = missing[missing > 0].sort_values(ascending=False)
plt.figure(figsize=(10, 5))
missing.plot(kind='bar', color='#1D9E75')
plt.title('% Missing Values by Column')
plt.xticks(rotation=45)
plt.show()

# Drop columns with >50% missing
df = df.drop(columns=missing[missing > 50].index)

# Impute numeric columns with median
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())

# Impute categorical columns with mode
categorical_cols = df.select_dtypes(include=['object']).columns
df[categorical_cols] =
df[categorical_cols].fillna(df[categorical_cols].mode().iloc[0])

# Flag trick for MNAR column (we'll simulate this)
df['total_cost_missing'] = df['total_cost'].isnull().astype(int)
```

```
# Verify no missing values left
```

```
print("Missing remaining:", df.isnull().sum().sum())
```

The Flag Trick

For MNAR columns, create a binary column that records whether the value was missing before you impute. This tells the model, "This value was hidden," which is often more useful than the imputed value alone.

3. Detecting & Treating Outliers

Outliers are extreme values that can distort models. Ask two questions: Is this a data entry error? Or a genuine extreme value?

For example, a tourist spending entry of 20,000,000 Tanzanian shillings is likely a typo. One of 2,000,000 might be a luxury safari booking worth keeping.

IQR Method

Calculate Q1 (25th percentile) and Q3 (75th percentile).
IQR = Q3 - Q1
Outlier if < Q1 - 1.5×IQR or > Q3 + 1.5×IQR

Z-Score Method

Calculate Z = (value - mean) / std
Outlier if |Z| > 3
Assumes normal distribution.

```
LAB 2: OUTLIERS● Run in Colab



import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


# Simulate Tanzania tourist spending data

np.random.seed(42)

spending = np.concatenate([

    np.random.normal(500000, 150000, 1000),

    [2000000, 2500000, 3000000]

])

df_spend = pd.DataFrame({'total_cost': spending})


# IQR detection
```

```
Q1 = df_spend['total_cost'].quantile(0.25)

Q3 = df_spend['total_cost'].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR

upper = Q3 + 1.5 * IQR


# Option A: Remove outliers

df_remove = df_spend[(df_spend['total_cost'] >= lower) &
(df_spend['total_cost'] <= upper)]


# Option B: Cap outliers

df_cap = df_spend.copy()

df_cap['total_cost'] = df_cap['total_cost'].clip(lower=lower, upper=upper)


# Option C: Log transform

df_log = df_spend.copy()

df_log['total_cost_log'] = np.log1p(df_log['total_cost'])


# Plot comparison

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.hist(df_spend['total_cost'], bins=30, color='#1D9E75', alpha=0.7)

ax1.set_title('Original Distribution')

ax2.hist(df_log['total_cost_log'], bins=30, color='#1D9E75', alpha=0.7)

ax2.set_title('Log-Transformed Distribution')

plt.tight_layout()
```

```
plt.show()
```

How to choose treatment

Remove: Only clear data entry errors

Cap: When extreme values are real but rare (e.g., luxury safaris)

Log transform: When data is right-skewed and model benefits from normality

4. Normalisation & Standardisation

Imagine comparing age (18–65) and monthly income (Ksh 50,000–5,000,000) in the same model. The income column would dominate calculations simply because its numbers are larger. Scaling puts all features on equal footing.

Min-Max Scaling

Rescales to [0, 1] range. Sensitive to outliers. Use when you need bounded values.

Standard Scaling

Mean = 0, Std = 1. The default choice for most models.

Log Transform

Use np.log1p() for right-skewed data (income, spending, counts).

Robust Scaling

Uses median and IQR. Best when many outliers exist.

```
LAB 3: SCALING● Run in Colab



import pandas as pd

from sklearn.preprocessing import StandardScaler, MinMaxScaler


# Create small Kenyan demo dataset

data = {

    'age': [25, 30, 35, 40, 45, 50, 55, 60],
```

```
    'income_ksh': [50000, 75000, 120000, 200000, 350000, 500000, 750000,
1200000]

}

df = pd.DataFrame(data)


# Split into train/test (simulated split)

X_train = df.iloc[:6]

X_test = df.iloc[6:]


# Standard Scaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# Min-Max Scaler

mm_scaler = MinMaxScaler()

X_train_mm = mm_scaler.fit_transform(X_train)

X_test_mm = mm_scaler.transform(X_test)


# Show results

print("--- Standard Scaled Train ---")


print(pd.DataFrame(X_train_scaled, columns=df.columns).describe())
```

Data Leakage Alert

Never fit the scaler on the full dataset or the test set. Fit only on X_train, then transform both X_train and X_test. If you fit on test data, you leak future information into your training process.

5. Encoding Categorical Variables

Machine learning models only understand numbers. We must convert text categories into numeric form. Choose the right method to avoid creating false patterns.

Label Encoding

Assigns integers: Low=0, Medium=1, High=2.
Safe only for ordinal categories with meaningful order. Fine for tree models. Creates false ordering for nominal data.

One-Hot Encoding

Creates binary columns: is_kenya, is_uganda, etc.
No false ordering. Required for linear models. Causes column explosion for high-cardinality features.

LAB 4: ENCODING ● Run in Colab

```
import pandas as pd


# Tourist survey data

data = {

    'country': ['Kenya', 'Uganda', 'Tanzania', 'Rwanda', 'Kenya', 'USA'],

    'purpose': ['Leisure', 'Business', 'Leisure', 'Conference', 'Leisure',
'Business'],

    'satisfaction': ['Low', 'Medium', 'High', 'Medium', 'High', 'Low'],

    'spending_USD': [800, 2200, 1500, 3000, 1200, 2800]

}

df = pd.DataFrame(data)


print("Original shape:", df.shape)


# Label encode ordinal column (satisfaction)

satisfaction_map = {'Low': 0, 'Medium': 1, 'High': 2}

df['satisfaction_enc'] = df['satisfaction'].map(satisfaction_map)


# One-hot encode nominal columns

df = pd.get_dummies(df, columns=['country', 'purpose'], drop_first=True)


print("\nShape after encoding:", df.shape)

print("\nEncoded DataFrame:")


print(df.head())
```

High-Cardinality Handling

If a column has 50+ unique values, don't use one-hot encoding. Instead: (1) Group rare categories into "Other", or (2) Use target encoding (Week 8). For now, group rare categories to keep column count manageable.

6. Feature Engineering

Feature engineering turns domain knowledge into model advantage. You create new columns that make patterns explicit.

Strategy 1: Date-Derived Features

From a single "arrival_date" column, you can extract trip duration, month, day of week, and peak season flags.

Strategy 2: Ratio Features

Cost per day, cost per person, activities per day. Ratios often capture behaviour better than raw counts.

Strategy 3: Binning

Turn continuous numbers into categories: budget, mid-range, premium. Useful when exact values matter less than tiers.

```
LAB 5: FEATURE ENGINEERING● Run in Colab



import pandas as pd

import numpy as np



# Tourism bookings data

data = {

    'arrival_date': ['2024-06-15', '2024-03-20', '2024-12-22', '2024-09-10',
'2024-07-01'],

    'departure_date': ['2024-06-22', '2024-03-25', '2024-12-30',
'2024-09-15', '2024-07-10'],

    'total_cost_USD': [2100, 850, 3200, 1100, 2700],

    'group_size': [2, 1, 4, 2, 3],

    'num_activities': [8, 3, 12, 5, 9]

}

df = pd.DataFrame(data)



# Convert dates

df['arrival_date'] = pd.to_datetime(df['arrival_date'])

df['departure_date'] = pd.to_datetime(df['departure_date'])



# 1. Date-derived features

df['trip_duration_days'] = (df['departure_date'] -
df['arrival_date']).dt.days

df['arrival_month'] = df['arrival_date'].dt.month
```

```
peak_months = [6, 7, 8, 12]

df['is_peak_season'] = df['arrival_month'].isin(peak_months).astype(int)


# 2. Ratio features

df['cost_per_day'] = df['total_cost_USD'] / df['trip_duration_days']

df['cost_per_person'] = df['total_cost_USD'] / df['group_size']

df['activities_per_day'] = df['num_activities'] / df['trip_duration_days']


# 3. Binning

df['spend_tier'] = pd.cut(df['total_cost_USD'],

                          bins=[0, 1000, 2500, np.inf],

                          labels=['budget', 'mid-range', 'premium'])



print(df.T)
```

7. Feature Selection

More features do not always equal better models. Too many features cause overfitting and slow training.

Method 1: Variance Threshold

Drop columns with near-zero variance. If 99% of rows have the same value, the column adds little information.

Method 2: Correlation Matrix

Drop one column from any pair with correlation r > 0.95. They carry almost identical information.

Method 3: Feature Importances

Use tree-based models to rank features by importance. We cover this fully in Week 8.

8. Complete Preprocessing Pipeline

Chaining steps manually is error-prone. Scikit-learn Pipeline and ColumnTransformer let you wrap all preprocessing into a single object you can fit once and reuse everywhere.

```
LAB 6: PIPELINE● Run in Colab



import pandas as pd

from sklearn.pipeline import Pipeline
```

```
from sklearn.compose import ColumnTransformer

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler, OneHotEncoder


# Simulate full dataset

data = {

    'age': [28, 35, 42, 55, 33, 48],

    'income': [120000, 250000, np.nan, 800000, 180000, 450000],

    'country': ['Kenya', 'Uganda', 'Kenya', 'Tanzania', 'Rwanda', 'Kenya'],

    'purpose': ['Leisure', 'Business', 'Leisure', 'Leisure', 'Conference',
'Business']

}

df = pd.DataFrame(data)

X = df


# Define column types

numeric_features = ['age', 'income']

categorical_features = ['country', 'purpose']


# Numeric pipeline

numeric_pipeline = Pipeline(steps=[

    ('imputer', SimpleImputer(strategy='median')),

    ('scaler', StandardScaler())

])


# Categorical pipeline

categorical_pipeline = Pipeline(steps=[
```

```
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first', sparse_output=False))
])

# Combine with ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_pipeline, numeric_features),
        ('cat', categorical_pipeline, categorical_features)
    ])

# Fit on train, transform train/test (simulated)
X_train = X.iloc[:4]
X_test = X.iloc[4:]

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("Pipeline ready. Pass preprocessor into any Scikit-learn model in Week 4.")
```

Why Pipelines Matter

Pipelines prevent data leakage, make code reproducible, and simplify production deployment. You save one object instead of remembering 10 separate transformation steps.

Resources

Official Documentation

Preprocessing Data

Scikit-learn

Free

Imputation of Missing Values

Scikit-learn

Free

Pipelines & ColumnTransformer

Scikit-learn

Free

Working with Missing Data

Pandas

Free

Interactive Courses

Data Cleaning

Kaggle Learn

Free

Feature Engineering

Kaggle Learn

Free

Preprocessing for ML in Python

DataCamp

Free via Ngao

Books & Deep Dives

Feature Engineering Chapter

Python Data Science Handbook

Free

End-to-End ML Project

Aurélien Géron, Chapter 2

Free

Feature Engineering for Machine Learning

Alice Zheng & Amanda Casari

Book

Africa Datasets for Practice

Tanzania Tourism Prediction

Zindi (this week's challenge)

`Africa`

Financial Inclusion in Africa

Zindi (revisit with new skills)

`Africa`

Kenya Open Data Portal

Kenya Government

`Africa`

Academic References

1. Rubin, D.B. (1976). Inference and missing data. Biometrika, 63(3), 581–592. doi.org/10.2307/2335739
2. Tukey, J.W. (1977). Exploratory Data Analysis. Addison-Wesley.
3. García, S., Luengo, J. & Herrera, F. (2015). Data Preprocessing in Data Mining. Springer.
4. Zheng, A. & Casari, A. (2018). Feature Engineering for Machine Learning. O'Reilly.
5. Pedregosa, F., et al. (2011). Scikit-learn: ML in Python. JMLR, 12, 2825–2830. jmlr.org/papers/v12/pedregosa11a.html
6. van Buuren, S. (2018). Flexible Imputation of Missing Data, 2nd ed. stefvanbuuren.name/fimd/